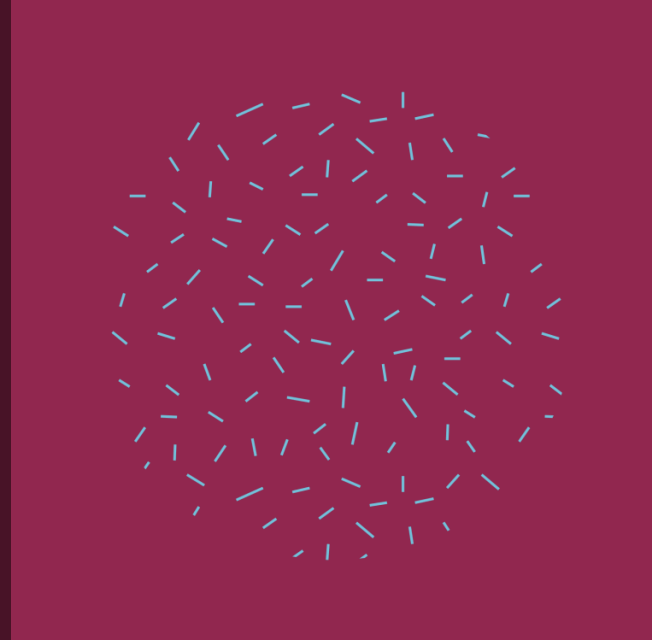
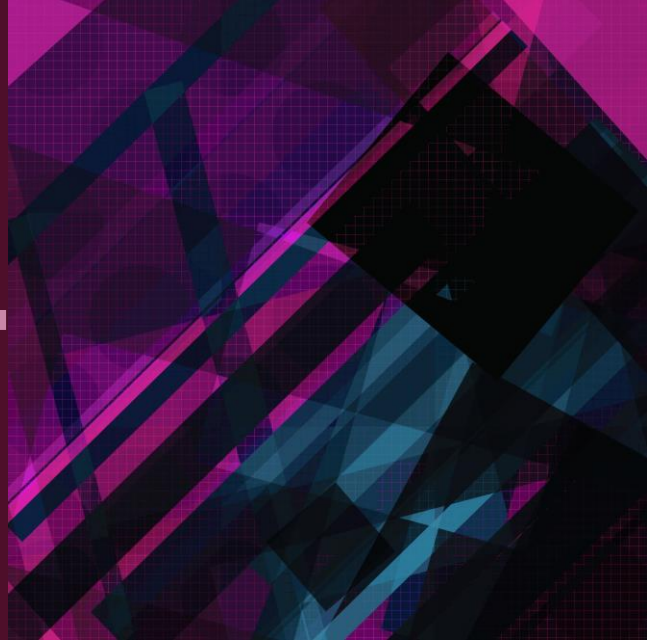




Data Analysis

Clustering

Section 6

Eng. Asmaa Fathy

Clustering

- Clustering is an unsupervised learning task in which an algorithm groups a set of objects in such a way that objects in the same group (called a cluster) are more similar to each other than to those in other groups (clusters). The goal of clustering is to discover the inherent groupings or structure in the data.
- PARTITION CLUSTERING
- HIERARCHICAL CLUSTERING
- DENSITY-BASED CLUSTERING
- GRID-BASED CLUSTERING
- PRINCIPAL COMPONENT ANALYSIS



PARTITION CLUSTERING

- It's a method in Machine Learning (without labels, i.e., unsupervised) where we divide data into groups (clusters).
- Each group contains points that are similar to each other, and each group is different from the others.
- The goal is to discover the "natural division" of data without anyone telling us the correct answer.
- What is it used for? Customer Segmentation, Image analysis, Anomaly Detection.

The two most important types of Clustering:

K-Means	K-Medoids
Each group has a center (a point in the middle). How does it work? 1. Choose the number of clusters (K). 2. Define initial centers. 3. Each point moves to the nearest center. 4. Update the centers. 5. Repeat until the result is stable. Affected by outliers	Similar to K-Means, but the difference: Instead of using a "center", it uses a real point in the data (medoid). Not Affected by outliers
Example, we have 5 points: 2, 3, 4, 10, 11 And we want to divide them into two groups (K = 2) 1. Choose initial positions: Let's assume: Position 1 = 2 and Position 2 = 10 2. Each point goes to the nearest position: (2 → 2), (3 → 2), (4 → 2), (10 → 10), (11 → 10) We now have: Group 1: (2, 3, 4) and Group 2: (10, 11) 3. Calculate the centroid (average): Group 1: $(2 + 3 + 4) / 3 = 3$ Group 2: $(10 + 11) / 2 = 10.5$ 4. Result: New centroids: 3, 10.5 5. Repeat (but nothing changes here) Center is average (3, 10.5)	Same data: 2, 3, 4, 10, 11 1. Choose centers (must be from the data itself): Assume: Center 1 = 2, Center 2 = 10 2. Distribute the points (same as K-Means): Group 1: (2, 3, 4) and Group 2: (10, 11) 3. Choose the best point to represent each group: Group (2, 3, 4) Calculate the sum of the distances: If we choose 2 = $2-3 + 2-4 = 1 + 2 = 3$ If we choose 3 = $3-2 + 3-4 = 1 + 1 = 2$ ✓ If we choose 4 = $4-2 + 4-3 = 2 + 1 = 3$ Best medoid = 3 Group (10, 11): 10 → $10-11 = 1$, 11 → $11-10 = 1$ Which one works (let's choose 10, for example) 4. Result: medoid 1 = 3 and medoid 2 = 10 Center is a data point (3, 10)

How do we determine the number of clusters?

Method 1: Elbow Method

- We don't know how many clusters (ks) to choose.
- So, we say: Let's try 1, 2, 3, 4...and see how the “error” decreases.
- What does Error mean? It's how far the points are from the center.
- The closer The points are to the center → The error is small (good).
- The farther the points are → The error is large (bad).

Example:

K = 1 → Error = 100

K = 2 → Error = 50

K = 3 → Error = 30

K = 4 → Error = 25

K = 5 → Error = 23

- From 1 → 2: Significant drop (100 → 50),
- From 2 → 3: Good drop (50 → 30),
- After that: The improvement becomes slight.
- Here, the "elbow" is at K = 3
- As soon as you add clusters, the quality improves quickly. Then, the improvement slows down.
- We choose point before the improvement slows down = the "elbow".

How do we determine the number of clusters?

Method 2: Silhouette Analysis

- It measures for each point: Is it comfortable in this group or not?
- How is it calculated? For each point, we calculate 3 things:
- We calculate (a): The average distance between it and the points in the same group.
- We calculate (b): The average distance between it and the next nearest group
- We calculate the final value: $s = \frac{b - a}{\max(a, b)}$
- If s is Close to 1 → Excellent
- If s is Close to 0 → Not clear
- If s is Negative → Wrong Group

Example We have:

Cluster 1: (2, 3, 4)

Cluster 2: (10, 11)

Let's take the point = 3

We calculate a: The distances between it and the same group: $|3 - 2| = 1$, $|3 - 4| = 1$.

Average of a = $(1 + 1) / 2 = 1$

Calculate b: Distances between it and the next nearest group (Cluster 2): $|3 - 10| = 7$, $|3 - 11| = 8$.

Average of b = $(7 + 8) / 2 = 7.5$

Calculate the Silhouette: $s = (7.5 - 1) / 7.5 = 0.87$ (Close to 1) → The point is in the correct group.

If we put point 10 wrong with (2, 3, 4) →

So Cluster 1: (2, 3, 4, 10)

Cluster 2: (11)

We calculate a: $|10 - 2| = 8$, $|10 - 3| = 7$, $|10 - 4| = 6$.

Average of a = $(8 + 7 + 6) / 3 = 7$

Calculate b: $|10 - 11| = 1$.

Calculate the Silhouette: $s = (1 - 7) / 7 = -6/7 = -0.86$ (negative) → The point is in the wrong group.

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns
from pyclustering.cluster.kmedoids import kmedoids
```

```
# creating 3 data sets (clusters):

# Generates 100 points (samples). Each point has: x and y (meaning 2 features)
# mean = 5 → Data center around 5 , std = 3 → Scatter around it
# Meaning cluster around (5, 5)
c1 = np.random.normal(5, 3,(100, 2))

# 100 new points, centered around 15, Larger dispersion (5), Cluster again around (15, 15)
c2 = np.random.normal(15, 5,(100, 2))

# 100 points, Centered at -5, Smaller dispersion (2), Cluster again around (-5, -5)
c3 = np.random.normal(-5, 2,(100, 2))

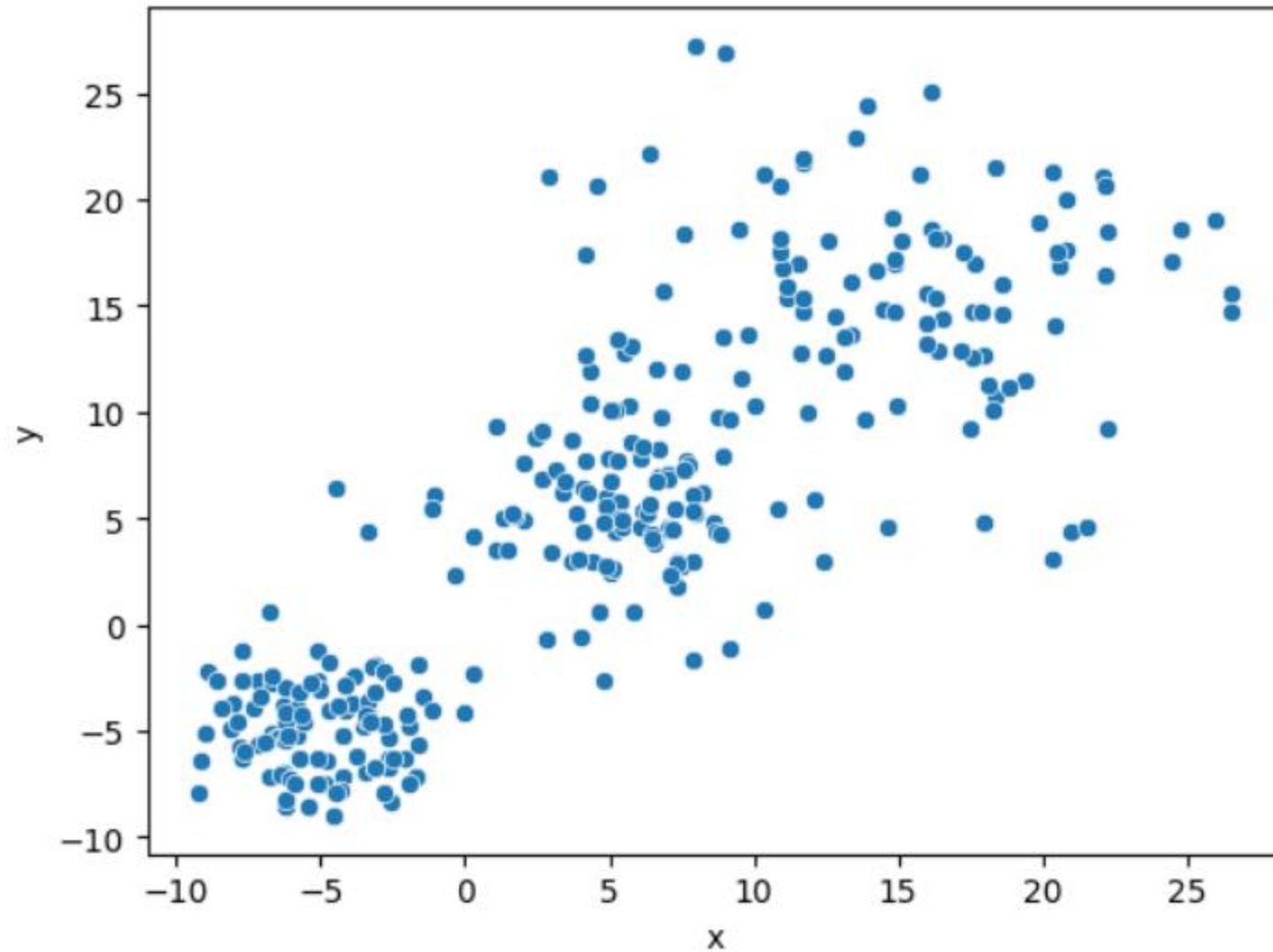
# Combines the 3 datasets into one table. Instead of 3 separate clusters
# One dataset with 300 points
d = np.concatenate((c1, c2, c3), axis = 0)

d.shape
```

```
(300, 2)
```

```
df = pd.DataFrame(d, columns=["x", "y"])
sns.scatterplot(data=df, x="x", y="y")
```

<Axes: xlabel='x', ylabel='y'>



Let's train a model using K-Means

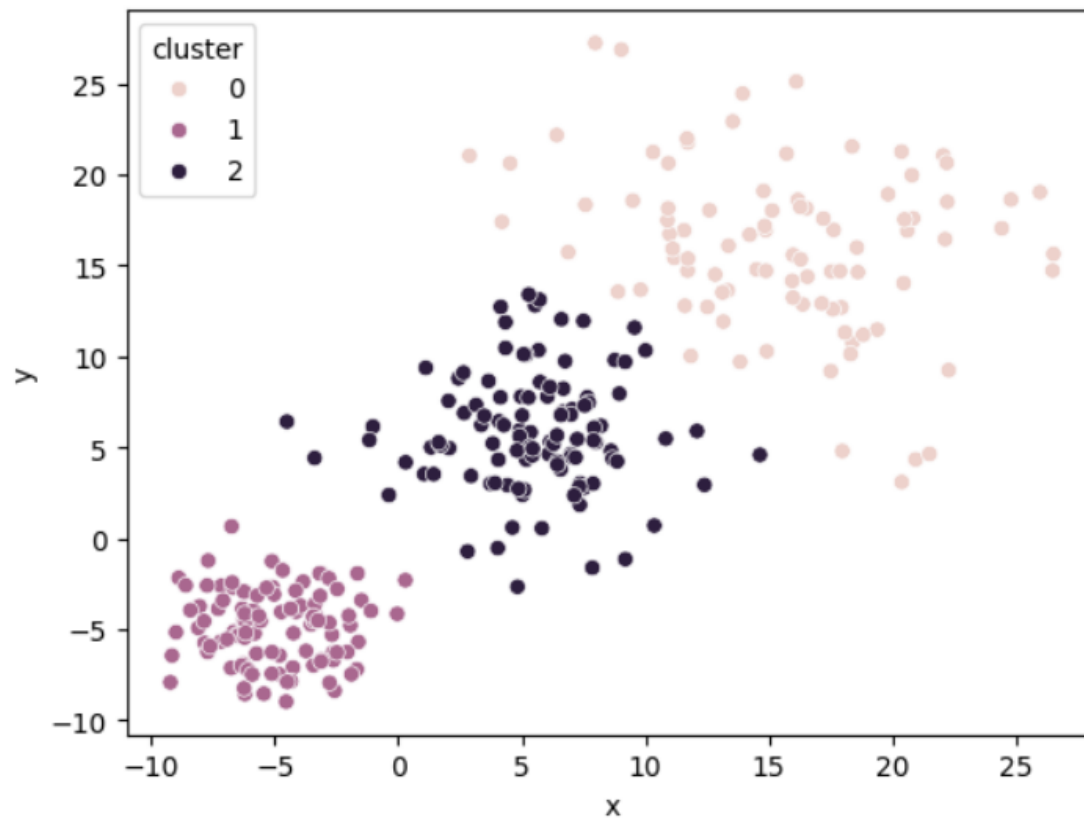
```
kmeans = KMeans(n_clusters=3, random_state=0).fit(d)
```

```
# random_state=0 --> This makes the result constant every time (the same starting point, so the results don't change with each run)
```

```
# `kmeans.labels_` --> Cluster number For each point
```

```
df["cluster"] = kmeans.labels_  
sns.scatterplot(data=df, x="x", y="y", hue="cluster")
```

```
<Axes: xlabel='x', ylabel='y'>
```



```
# Choosing Initial Medoids:  
# You choose 3 points from the data as a starting point  
initial_medoids = [0, 100, 200]
```

```
# Creating a KMedoids Model:  
kmedoids_instance = kmedoids(d, initial_medoids)
```

```
# Running the Algorithm  
kmedoids_instance.process()
```

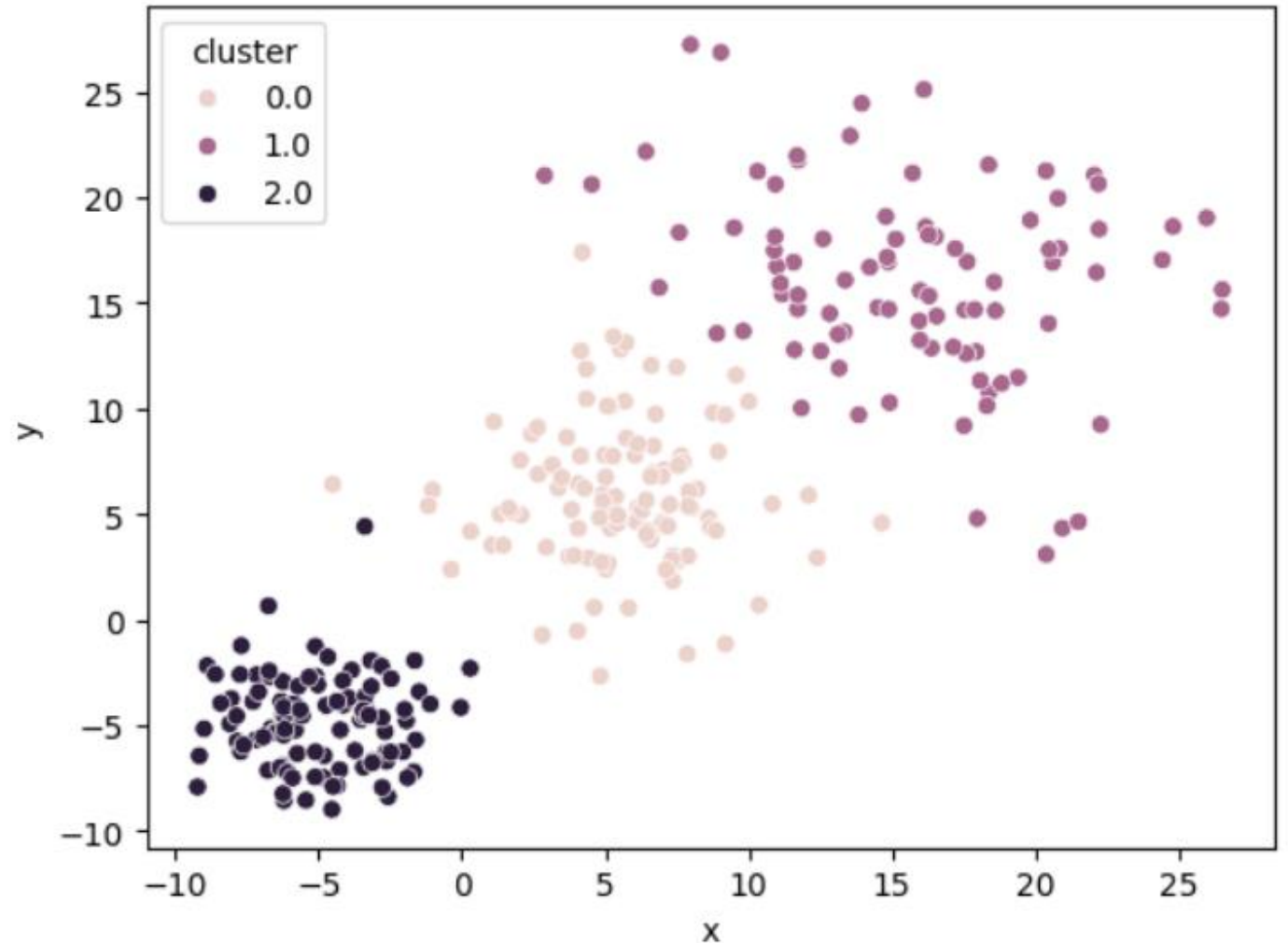
```
<pyclustering.cluster.kmedoids.kmedoids at 0x20a7ad441d0>
```

```
# Extracting the Clusters  
clusters = kmedoids_instance.get_clusters()
```

```
# Creating labels for each point  
labels = np.zeros(len(d))  
  
for cluster_id, cluster in enumerate(clusters):  
    for idx in cluster:  
        labels[idx] = cluster_id
```

```
df = pd.DataFrame(d, columns=["x", "y"])  
df["cluster"] = labels  
  
sns.scatterplot(data=df, x="x", y="y", hue="cluster")
```

```
<Axes: xlabel='x', ylabel='y'>
```



Let's train a model using K-Medoids

Conclusion: K-Means and K-Medoids did
equally well for this dataset.

Let Us Add One Outlier to the Dataset

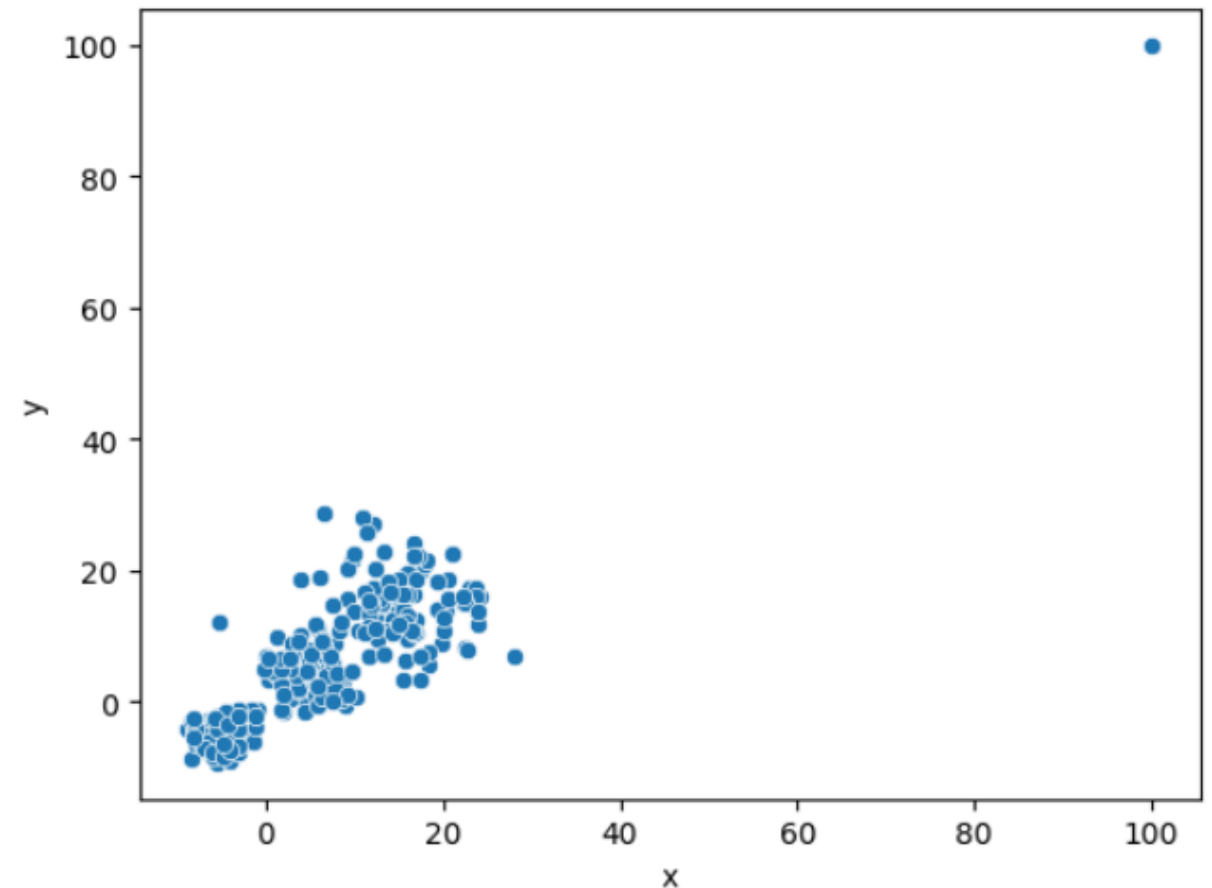
```
c1 = np.random.normal(5, 3,(100, 2))  
c2 = np.random.normal(15, 5,(100, 2))  
c3 = np.random.normal(-5, 2,(100, 2))
```

```
outlier = np.array([[100, 100]])
```

```
d = np.concatenate((c1, c2, c3, outlier), axis = 0)
```

```
df = pd.DataFrame(d, columns=["x", "y"])  
sns.scatterplot(data=df, x="x", y="y")
```

<Axes: xlabel='x', ylabel='y'>



Train a model using K-Means

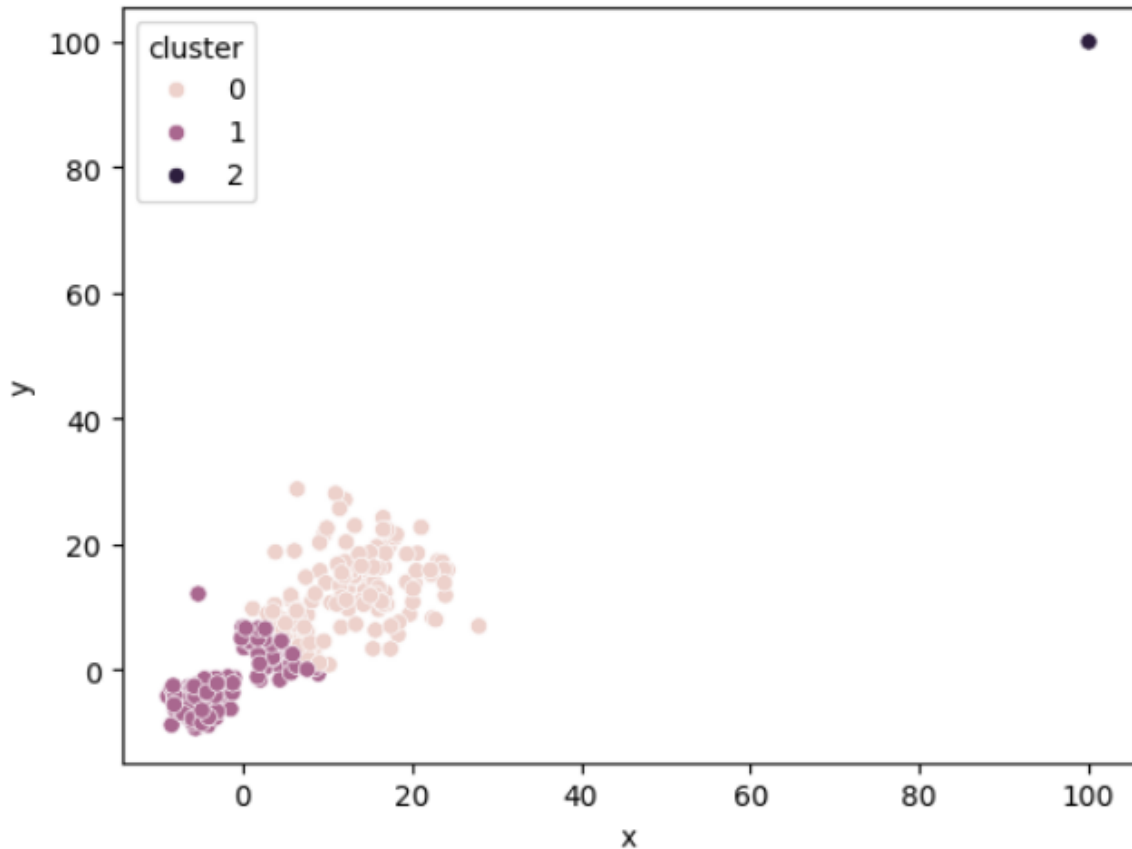
```
df = pd.DataFrame(d, columns=["x", "y"])

kmeans = KMeans(n_clusters=3, random_state=0).fit(d)

df["cluster"] = kmeans.labels_

sns.scatterplot(data=df, x="x", y="y", hue="cluster")
```

<Axes: xlabel='x', ylabel='y'>



Train a model using K-Medoids

```
initial_medoids = [0, 100, 200]

kmedoids_instance = kmedoids(d, initial_medoids)
kmedoids_instance.process()

clusters = kmedoids_instance.get_clusters()

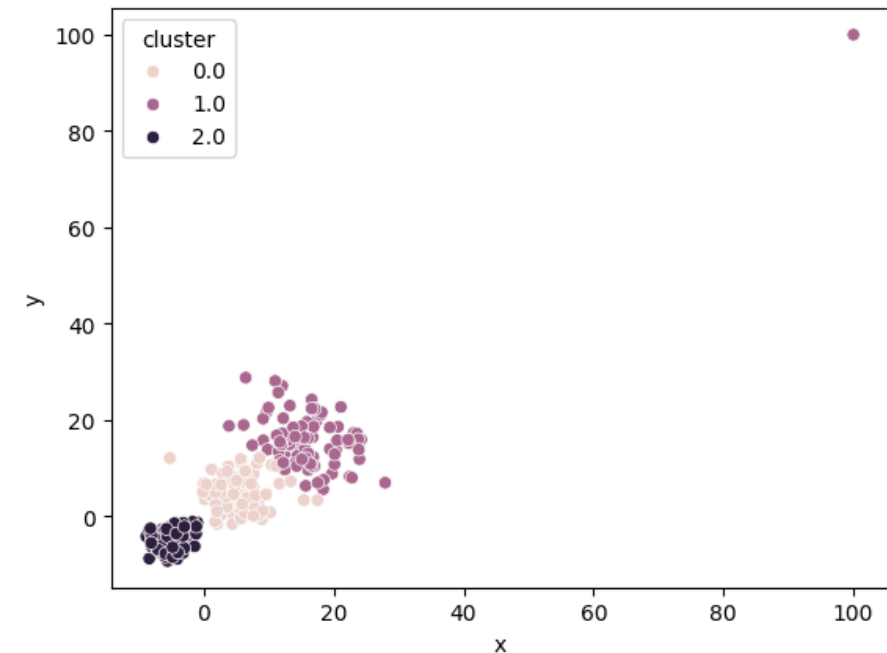
labels = np.zeros(len(d))

for cluster_id, cluster in enumerate(clusters):
    for idx in cluster:
        labels[idx] = cluster_id

df = pd.DataFrame(d, columns=["x", "y"])
df["cluster"] = labels

sns.scatterplot(data=df, x="x", y="y", hue="cluster")
```

<Axes: xlabel='x', ylabel='y'>



Conclusion:
K-Medoids
did well
even with
an outlier.
K-Means
failed.

HIERARCHICAL CLUSTERING

Imagine this: You have 4 points:

A B C D

Initially, Each one separately

Step 1: The two closest points are brought together

For example:

A is close to B

C is close to D

So: (A,B) (C,D)

Step 2: The assembly continues then the two groups get closer to each other:

(A,B,C,D)

This is what a Dendrogram looks like:

```
      (A,B,C,D)
      /   \
    (A,B) (C,D)
    /\   /\
   A  B  C  D
```

- This is an unsupervised learning method (meaning no labels) whose goal is to discover clusters within the data and to understand the relationships between them in a hierarchical way (like a tree).
- Hierarchical clustering: Create a tree called a dendrogram. This is a tree illustrating how the data was assembled step by step.

You're cutting the tree at a specific level:

If you cut above:

(A, B, C, D)

Result = 1 cluster (all together)

If you cut here:

```
      -----
      |       |
    (A,B) (C,D)
```

Result = 2 clusters

Cluster 1: A, B

Cluster 2: C, D

If you cut below like this :

A B C D

Result = 4 clusters (each point separately)

The Dendrogram is a map that shows you the stages of clustering, and you decide: How many clusters do you want? Depending on where you cut.

Two Types of Hierarchical Clustering

Agglomerative (Bottom-up)	Divisive (Top-down)
We start with each point individually. We gradually merge them.	We start with a large group and divide it.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

# Create a fake dataset:
# n_samples=100 → You have 100 data points
# centers=4 → There are 4 clusters
# n_features=2 → Each point has two columns (Feature1, Feature2)
# random_state=42 → So that the same data is displayed each time (consistency)
# cluster_std=1.5 → The spread of points around the center (the larger the spread, the more scattered the cluster becomes)
# X = Coordinates (Feature1, Feature2)
# y = The actual cluster number (0, 1, 2, 3)

X, y = make_blobs(n_samples=100, centers=4, n_features=2, random_state=42, cluster_std=1.5)

# Convert to DataFrame for easier manipulation
df = pd.DataFrame(data=np.c_[X, y], columns=['Feature1', 'Feature2', 'Cluster'])

# Explore the dataset
print(df.head())
print(df.describe())
print(df.info())

```

```

   Feature1  Feature2  Cluster
0 -10.108518  5.051252      3.0
1  -5.659351 -8.726406      2.0
2  -3.213409  9.828126      0.0
3  -4.775436 -8.982886      2.0
4  -8.301647  8.164700      3.0

```

	Feature1	Feature2	Cluster
count	100.000000	100.000000	100.000000
mean	-3.582871	2.901643	1.500000
std	5.268073	6.293456	1.123666
min	-10.904832	-8.982886	0.000000
25%	-8.161044	-1.552449	0.750000
50%	-4.751208	4.685694	1.500000
75%	0.072281	8.205229	2.250000
max	6.856720	11.792703	3.000000

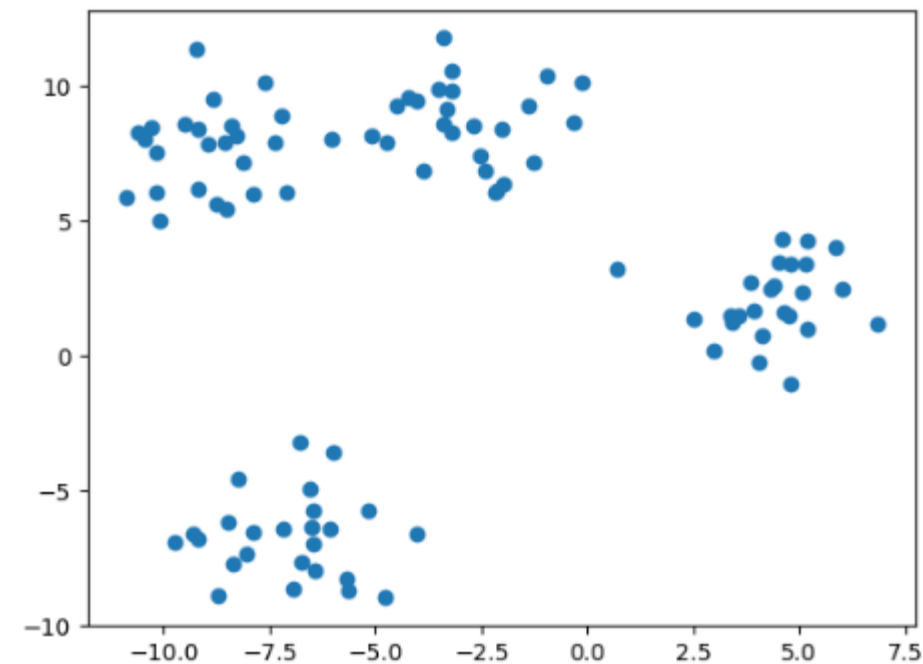
```

<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   Feature1  100 non-null    float64
1   Feature2  100 non-null    float64
2   Cluster   100 non-null    float64
dtypes: float64(3)
memory usage: 2.5 KB
None

```

```
plt.scatter(df['Feature1'], df['Feature2'])
```

```
<matplotlib.collections.PathCollection at 0x26cc4473c90>
```



```

# Standardize the features
# So that each feature falls within the same range (very important in clustering)
scaler = StandardScaler()

# fit_transform → Learns from the data and applies it
X_scaled = scaler.fit_transform(X)

```

```
# Testing the Number of Clusters from 2 to 7 clusters
num_clusters_range = range(2, 8)

# Perform agglomerative hierarchical clustering with different numbers
# A Loop iterates through each cluster
for num_clusters in num_clusters_range:

    # We create a clustering model: Starts with each point individually and then merges them (hierarchical)
    agglomerative = AgglomerativeClustering(n_clusters=num_clusters)

    # It does two things: Learns and returns the cluster number for each point
    agglomerative_labels = agglomerative.fit_predict(X_scaled)

    # Prints: Number of clusters and which cluster each point went to?
    print(f"\nAgglomerative Hierarchical Clustering with {num_clusters} Clusters:")
    print(f"Cluster Assignments: {agglomerative_labels}")

    # Evaluating the Score
    # Silhouette Score measures: Is the point close to its cluster? Is it far from the other clusters?
    # Result From -1 to 1. Everything closer to 1 is better
    silhouette_avg = silhouette_score(X_scaled, agglomerative_labels)
    print(f"Silhouette Score: {silhouette_avg:.4f}\n")
```

- Your results:

Clusters	Score
2	0.5268
3	0.6894
4	0.6966 ★
5	0.5915
6	0.5058
7	0.4200

- Best value = 4 clusters
- You already created the data with centers = 4
- So it works perfectly

Agglomerative Hierarchical Clustering

Agglomerative Hierarchical Clustering with 2 Clusters:

Cluster Assignments: [0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0 0
0 0 0 0 0 1 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0
1 0 1 0 1 1 0 1 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0]

Silhouette Score: 0.5268

Agglomerative Hierarchical Clustering with 3 Clusters:

Cluster Assignments: [0 1 0 1 0 1 2 2 2 0 0 1 1 2 0 0 0 2 0 0 0 0 2 2 0 2 0 0 1 1 2 1 0 0 0 0
0 2 2 0 2 1 0 0 1 1 0 0 2 2 2 2 1 0 0 0 0 2 0 1 2 0 1 0 0 1 0 2 1 2 1 0 0
1 0 1 0 1 1 2 1 0 0 0 0 0 1 0 2 2 1 0 1 0 0 0 2 0 2]

Silhouette Score: 0.6894

Agglomerative Hierarchical Clustering with 4 Clusters:

Cluster Assignments: [3 0 1 0 3 0 2 2 2 1 1 0 0 2 1 3 3 2 3 3 1 3 3 2 2 1 2 3 3 0 0 2 0 3 1 1 1
1 2 2 3 2 0 1 1 0 0 3 3 2 2 2 0 3 3 1 1 2 1 0 2 3 0 3 3 0 3 2 0 2 0 3 1
0 3 0 3 0 0 2 0 1 1 1 1 0 1 2 2 0 1 0 1 1 1 2 3 2]

Silhouette Score: 0.6966

Agglomerative Hierarchical Clustering with 5 Clusters:

Cluster Assignments: [1 3 0 3 1 4 2 2 2 0 0 3 3 2 0 1 1 2 1 1 0 1 1 2 2 0 2 1 1 4 4 2 3 1 0 0 0
0 2 2 1 2 3 0 0 4 4 1 1 2 2 2 2 3 1 1 0 0 2 0 3 2 1 4 1 1 3 1 2 3 2 3 1 0
3 1 3 1 3 4 2 4 0 0 0 0 0 4 0 2 2 3 0 3 0 0 0 2 1 2]

Silhouette Score: 0.5915

Agglomerative Hierarchical Clustering with 6 Clusters:

Cluster Assignments: [1 3 5 3 1 4 0 0 0 5 5 3 3 0 2 1 1 0 1 1 5 1 1 0 0 5 0 1 1 4 4 0 3 1 2 5 2
5 0 0 1 0 3 5 5 4 4 1 1 0 0 0 0 3 1 1 5 2 0 5 3 0 1 4 1 1 3 1 0 3 0 3 1 2
3 1 3 1 3 4 0 4 2 5 2 2 2 4 5 0 0 3 5 3 5 5 2 0 1 0]

Silhouette Score: 0.5058

Agglomerative Hierarchical Clustering with 7 Clusters:

Cluster Assignments: [0 1 5 1 0 4 6 6 6 5 5 1 1 3 2 0 0 6 0 0 5 0 0 6 6 5 3 0 0 4 4 6 1 0 2 5 2
5 6 3 0 3 1 5 5 4 4 0 0 3 3 6 6 1 0 0 5 2 6 5 1 3 0 4 0 0 1 0 6 1 3 1 0 2
1 0 1 0 1 4 6 4 2 5 2 2 2 4 5 3 6 1 5 1 5 5 2 6 0 3]

Silhouette Score: 0.4200

DENSITY-BASED CLUSTERING

- Instead of dividing the data into a specific number of clusters (like KMeans),
- We look for: **Where are the areas with many points close together? These areas = Cluster, Empty or few areas = Noise**

The most popular algorithm: DBSCAN (Density-Based Spatial Clustering)

If there's a point with many people nearby → it's part of a cluster

If it's alone → it's noise

1. It detects unusual shapes not necessarily circles like Kmeans. It can create clusters like: U-shapes, Lines, Irregular shapes
2. Noise is automatically generated any distant point is classified as -1
3. You don't need to specify the number of clusters Unlike KMeans

Important parameters:

1. **eps (epsilon):** This is the distance we use to ask "Is the point near or far?
Very small → Each point is separate
Very large → It's all one cluster
2. **min_samples:** The minimum number of points around a point for it to be a cluster.
For example: If = 5, then there must be 5 points close to it

Point Types in DBSCAN:

1. **Core Point:** A point surrounded by a sufficient number of points (min_samples)
2. **Border Point:** Close to the Core but not alone enough
3. **Noise:** Completely alone

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
```

```
# Generate sample data
```

```
centers = [[1, 1], [-1, -1], [1, -1]]
```

```
# These are the locations of the cluster centers. This means you have 3 clusters around: (1, 1), (-1, -1), (1, -1)
```

```
X, labels_true = make_blobs(
    n_samples=750, centers=centers, cluster_std=0.4, random_state=0
)
```

```
# create the data: 750 points
```

```
# centers=3 → 3 clusters
```

```
# cluster_std=0.4 → The points are close to the center (clear clusters)
```

```
# labels_true → This is the true classification
```

```
X = StandardScaler().fit_transform(X)
```

```
# We perform scaling (standardizing the values)
```

```
# This is very important in DBSCAN because the distances remain correct
```

```

# Running DBSCAN
db = DBSCAN(eps=0.3, min_samples=10).fit(X)
# `eps=0.3` → The distance at which we consider points to be "close"
# `min_samples=10` → There must be 10 points around the point for it to be a Core
# `.fit(X)` = Training the model on the data

# Defining the Core Points
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
# We created a False array with the same length as the number of points

core_samples_mask[db.core_sample_indices_] = True
# Any point from the Core Points → set to True
# db.core_sample_indices_: These are the indices (positions) of the points that are Core

# Get the labels
labels = db.labels_
# Each point has a cluster number
# For example: 0 → cluster 1, 1 → cluster 2, -1 → Noise

# Counting the Number of Clusters
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
# `set(labels)` → Find the different values (clusters)
# If there is `-1` (noise) → Remove it because noise is not a cluster

# Counting the Number of Noise Points
n_noise_ = list(labels).count(-1)

print("Estimated number of clusters: %d" % n_clusters_) # 3 --> 100% Correct
print("Estimated number of noise points: %d" % n_noise_) # 18

```

Estimated number of clusters: 3

Estimated number of noise points: 18

```
# Performance Evaluation this compares to `labels_true`

# 1- Homogeneity: Does each cluster contain only one type?
print("Homogeneity: %0.3f" %metrics.homogeneity_score(labels_true, labels)) # 0.953 --> Excellent

# 2- Completeness: Are all the points for the same class in the same cluster?
print("Completeness: %0.3f" %metrics.completeness_score(labels_true, labels)) # 0.883 --> Very Good

# 3- V-measure: Average of the two above
print("V-measure: %0.3f" %metrics.v_measure_score(labels_true, labels)) # 0.917 --> Excellent

# 4- Adjusted Rand Index (ARI): Similarity between actual and expected values
print("Adjusted Rand Index: %0.3f" %metrics.adjusted_rand_score(labels_true, labels)) # 0.952 --> Almost identical

# 5- Adjusted Mutual Information (AMI): Another similarity measure
print("Adjusted Mutual Information: %0.3f"%metrics.adjusted_mutual_info_score(labels_true, labels)) # 0.916 --> Very high

# 6- Silhouette Score: Are the points Close to their cluster or far from other clusters?
print("Silhouette Coefficient: %0.3f" %metrics.silhouette_score(X, labels)) # 0.626 = Good
```

Homogeneity: 0.953

Completeness: 0.883

V-measure: 0.917

Adjusted Rand Index: 0.952

Adjusted Mutual Information: 0.916

Silhouette Coefficient: 0.626

```

# Plot result
import matplotlib.pyplot as plt

# Specifying the different clusters
unique_labels = set(labels)
# We get all the different values in `labels`:
# For example: {0, 1, 2, -1} | 0, 1, 2 → clusters, -1 → noise

# Specifying colors for each cluster
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
# `np.linspace(0,1,...)` → Numbers from 0 to 1
# `plt.cm.Spectral` → Converts the number to a color
# This means each cluster has a color

# We loop through: `k` = Cluster number (0, 1, 2, -1), `col` = Its color
for k, col in zip(unique_labels, colors):

    # If cluster = -1, this is noise, let's make it black
    if k == -1:
        col = [0, 0, 0, 1]

    # This is a mask (True/False): True for the points in this cluster and False for the rest
    class_member_mask = labels == k

    # Drawing Core Points:

    # We select the points in cluster k and also the core points
    xy = X[class_member_mask & core_samples_mask]

    # Core Points appear large
    plt.plot(
        xy[:, 0],
        xy[:, 1],
        "o",
        markerfacecolor=tuple(col),
        markeredgecolor="k",
        markersize=14,
    )

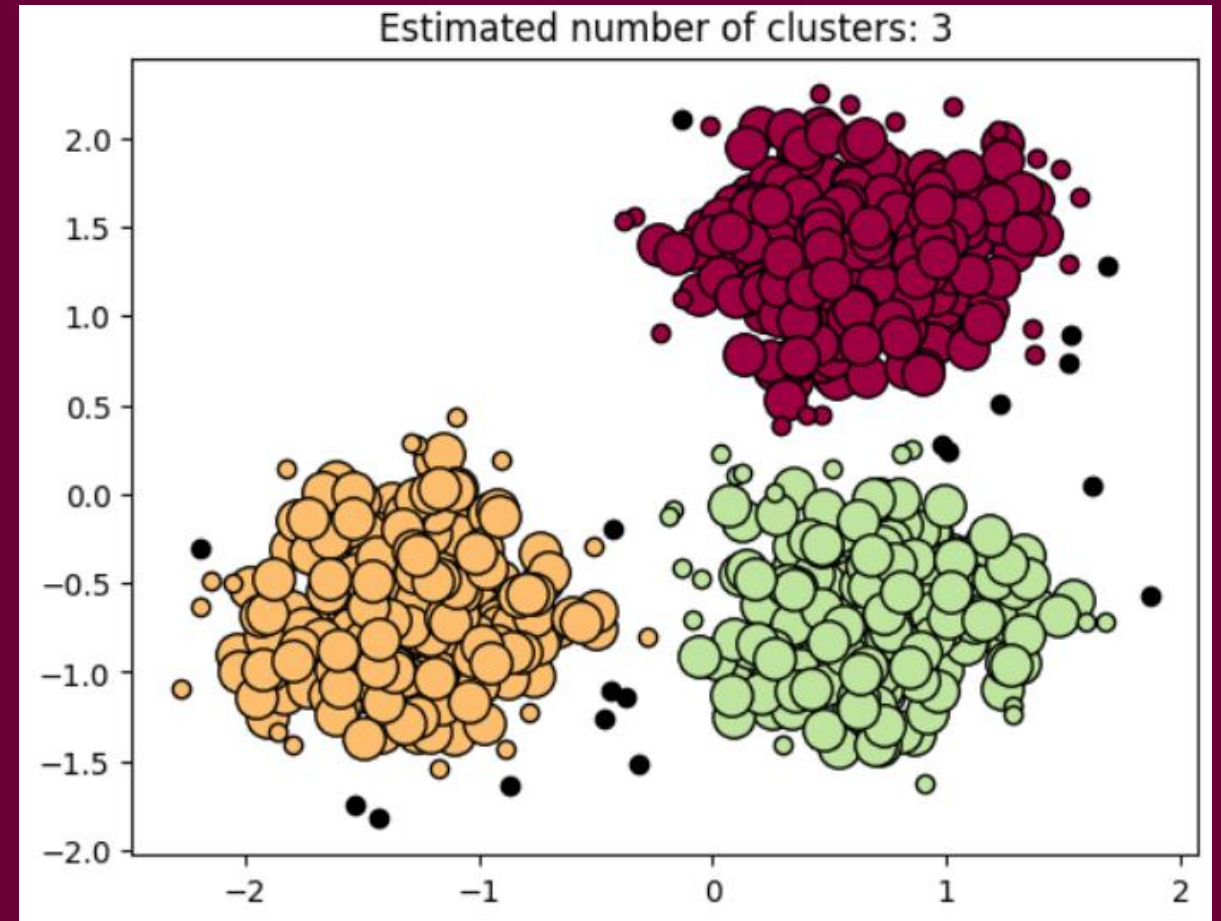
    # Drawing Border Points

    # Border is in the cluster but not a core
    xy = X[class_member_mask & ~core_samples_mask]

    # Small Border Points
    plt.plot(
        xy[:, 0],
        xy[:, 1],
        "o",
        markerfacecolor=tuple(col),
        markeredgecolor="k",
        markersize=6,
    )

plt.title("Estimated number of clusters: %d" % n_clusters_)
plt.show()

```



GRID-BASED CLUSTERING

- We divide the feature space into a grid then we see:
- Squares with many points → Cluster
- Empty ones → We leave them as they are

Imagine you have a map that you divide into squares.

Each square:

If it's full of people → Important area (Cluster)

If it's empty → Nothing there

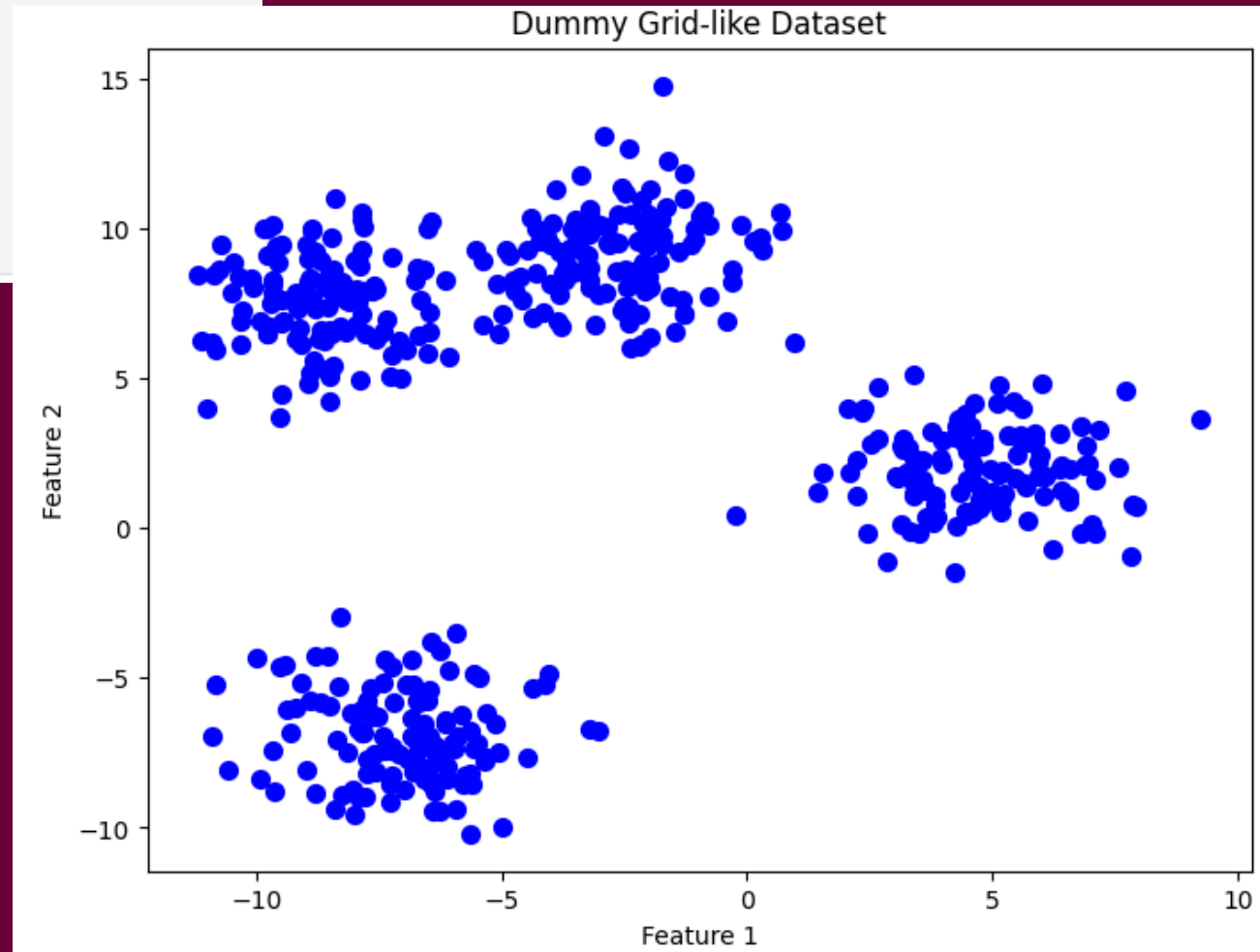
Most Popular Methods:

STING	OPTICS (Ordering Points To Identify Clustering Structure)	CLIQUE
• Divides data into grids. • Calculates statistics for each grid: Number of points, Average, and determines which grids constitute a cluster. • It depends on the grid size and Density (number of points). • The boundaries are squares (not flexible like DBSCAN)	• It's like DBSCAN, but instead of immediately identifying clusters, it first organizes the data. • It arranges points according to density then we discover clusters from this arrangement. Imagine: Cluster A: People very crowded together Cluster B: People slightly spaced out Cluster C: People standing alone • DBSCAN might confuse them • OPTICS understands both correctly	• Divides each feature into individual squares (Like a grid) • Experiments with combinations of features (Subspaces). • Looks for areas of high density → these are clusters Imagine data in the form of dots: • CLIQUE divides the space into squares then, squares with a lot of dots → cluster, Empty squares → ignore them. • If the data contains more than one feature (many columns). For example: age, income, height, weight. If you use them all at once, the clusters might not be clear; instead, look at Subspaces Like: (age, income), (age, height), (income, weight).


```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate a dummy dataset with a grid-like structure
X, y = make_blobs(n_samples=500, centers=4, cluster_std=1.5, random_state=42)

# Plot the dataset
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c='b', s=50)
plt.title("Dummy Grid-like Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Perform CLIQUE clustering

```
from pyclustering.cluster.clique import clique, clique_visualizer

# Perform CLIQUE clustering

# create CLIQUE algorithm for processing
# defines amount of cells in grid in each dimension
intervals = 20

# Lets consider each point as non-outlier
threshold = 3
clique_instance = clique(X.tolist(), intervals, threshold)

# start clustering process and obtain results
clique_instance.process()

# allocated clusters
clusters = clique_instance.get_clusters()
noise = clique_instance.get_noise()

# CLIQUE blocks that forms grid
cells = clique_instance.get_cells()

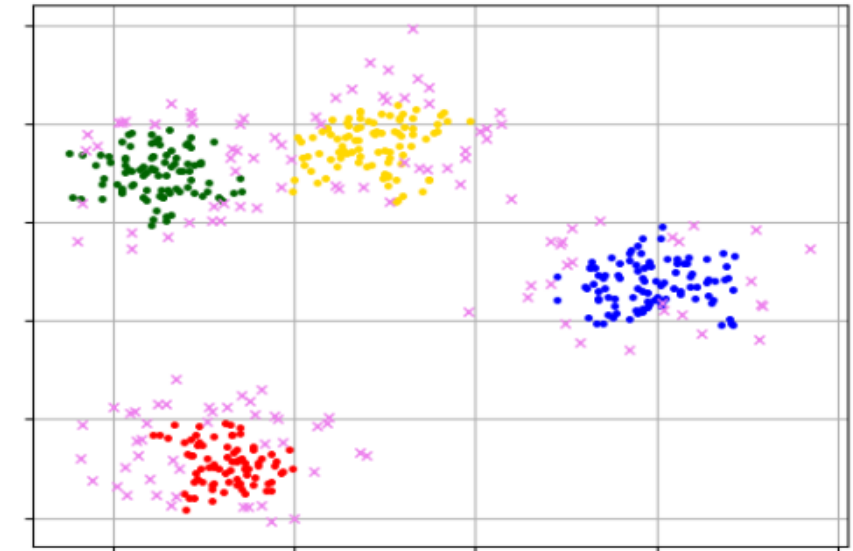
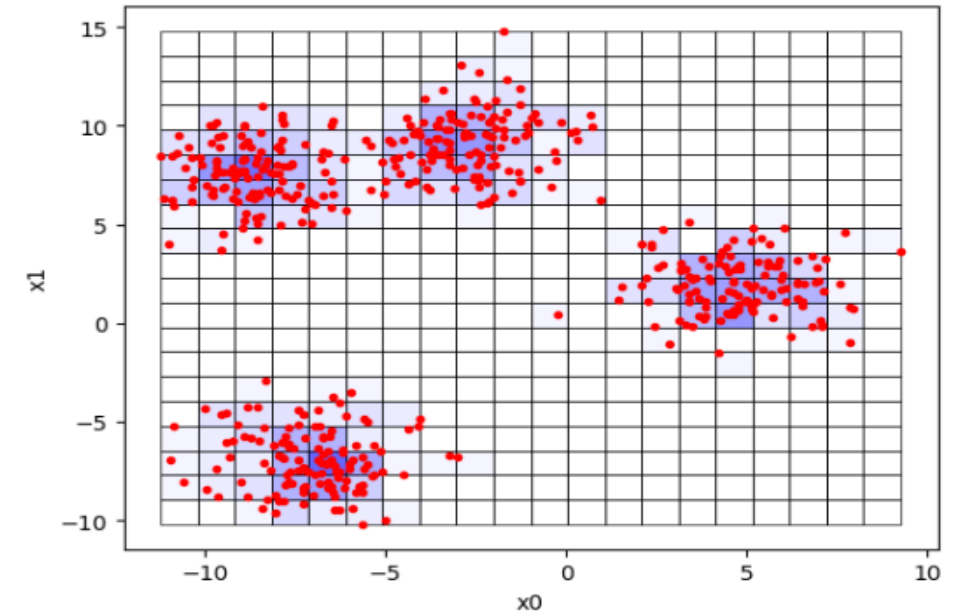
# points that are considered as outliers (in this example should be empty)
print("Amount of clusters:", len(clusters))

# visualize clustering results

# show grid that has been formed by the algorithm
clique_visualizer.show_grid(cells, X.tolist())

# show clustering results
clique_visualizer.show_clusters(X.tolist(), clusters, noise)
```

Amount of clusters: 4



PRINCIPAL COMPONENT ANALYSIS

This is a method used when we have data with many features, and we try to reduce them to a smaller number without losing important information.

PCA doesn't choose features but creates new features

Imagine you have data that includes Height, Weight, Age, Income, etc. PCA converts them into new axes instead of the old ones.

What are Principal Components? These are new features called:

- PC1 → The most important direction in the data (largest variance)
- PC2 → The second most important direction (perpendicular to the first)
- PC3 → and so on

- **Principal Component 1 (PC1) = General Trend**
 - Example: The Basic Relationship (Studying → Grades). The more you study, the higher your grade. Meaning "the student's overall excellence."
- **Principal Component 2 (PC2)= The Difference/Deviation from this Relationship**, but it does see small differences between students. For example:
 - A student studies little but gets higher than expected grades
 - Or a student studies a lot but gets lower than expected grades
 - So PC2 represents "Is the student performing better or worse than expected compared to their studying?"

Number-based vs. Variance-based PCA

Number-based PCA	Variance-based PCA
Decide the Number of components	keep a percentage of Information
How many features do you want?	How much information do I want to preserve?
Example: PCA(n_components=2) Keep the 2 most important ones and lose the rest	Example: PCA(0.95) Retain 95% of the data information (variance) It sorts the components by importance → then starts by taking: PC1 (most important), PC2, PC3, ... → until it reaches 95% of the information → then it stops
You might lose important information unintentionally	Preserves the most important data

The more components you reduce, the more information you lose, and the model gets corrupted.

Using Digits Dataset

```
from sklearn.datasets import load_digits
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
# Here we load a ready-made dataset called digits: These are images of numbers from 0 to 9, but not ordinary images. these are 8x8 pixels.
dataset = load_digits()
```

```
# This tells you what's inside the dataset
dataset.keys()
```

```
# `data` → The numbers themselves (pixels)
# `target` → The actual number (0, 1, 2...)
# `images` → The image as it appears
# `feature_names` → The column names
```

```
dict_keys(['data', 'target', 'frame', 'feature_names', 'target_names', 'images', 'DESCR'])
```

```
dataset.data.shape
```

```
# 1797 images
# Each image has 64 values
# Because the image is 8 x 8 = 64
```

```
(1797, 64)
```

```
dataset.data[0]
```

```
# This is the first number in the data
# It's an array with 64 numbers
# Each number = pixel intensity
```

```
array([ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.,  0.,  0., 13., 15., 10.,
        15.,  5.,  0.,  0.,  3., 15.,  2.,  0., 11.,  8.,  0.,  0.,  4.,
        12.,  0.,  0.,  8.,  8.,  0.,  0.,  5.,  8.,  0.,  0.,  9.,  8.,
         0.,  0.,  4., 11.,  0.,  1., 12.,  7.,  0.,  0.,  2., 14.,  5.,
        10., 12.,  0.,  0.,  0.,  0.,  6., 13., 10.,  0.,  0.,  0.]
```

```
# Converting it to an image
dataset.data[0].reshape(8,8)
```

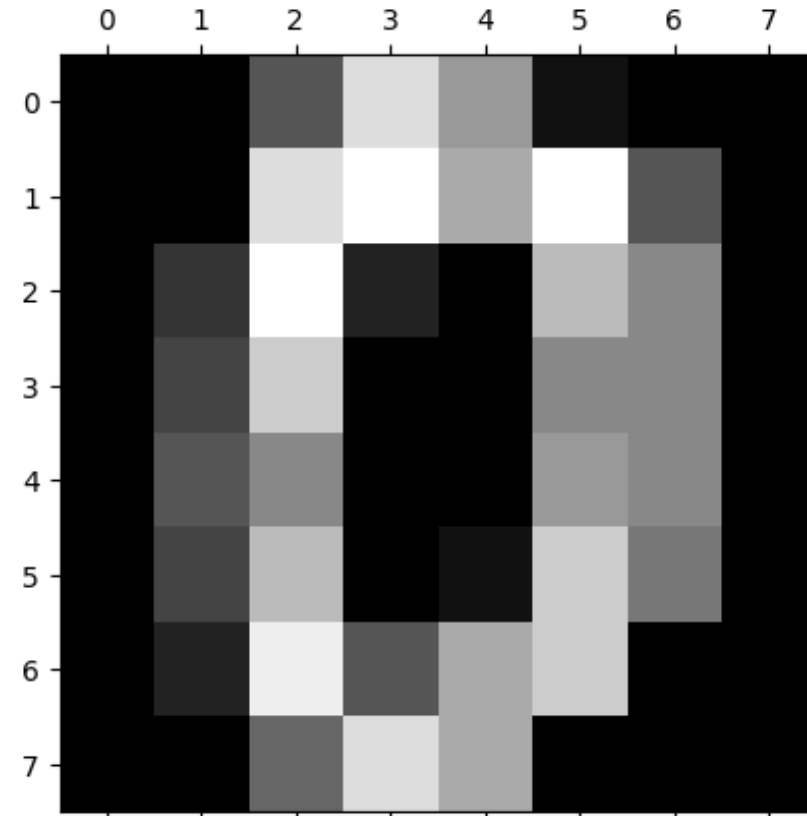
```
# we restore the original image shape Instead of: a single row with 64 numbers
# It becomes An 8x8 array. That is, an image
```

```
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])
```

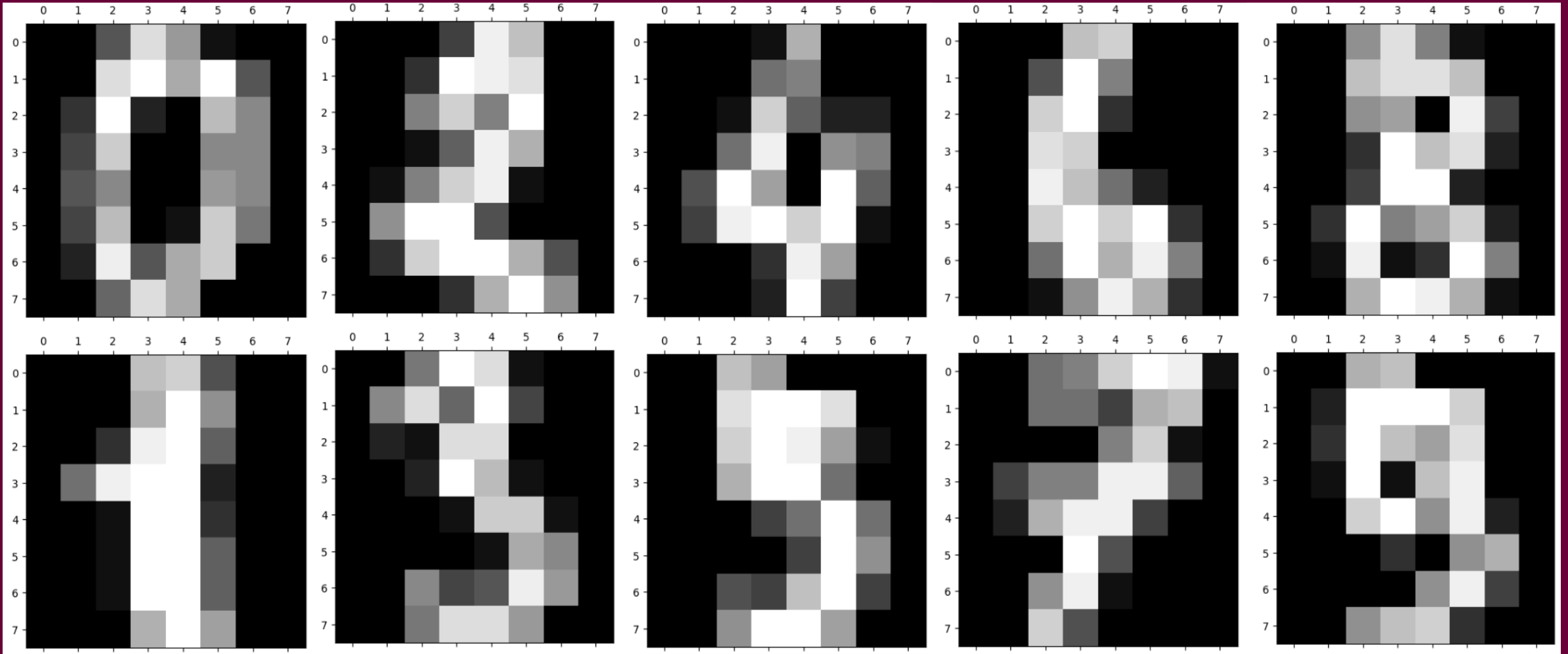
```
# Display Image
plt.gray() # Black and white
plt.matshow(dataset.data[0].reshape(8,8)) # Reshapes the image
```

<matplotlib.image.AxesImage at 0x26cc972d650>

<Figure size 640x480 with 0 Axes>



```
# Displays the first 10 images  
for i in range(10):  
    plt.matshow(dataset.data[i].reshape(8,8))
```



```
# This is an array containing the actual numbers
dataset.target
```

```
array([0, 1, 2, ..., 8, 9, 8], shape=(1797,))
```

```
# Convert to DataFrame
df=pd.DataFrame(dataset.data,columns=dataset.feature_names)
df.head()
```

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7	pixel_1_0	pixel_1_1	...	pixel_6_6	pixel_6_7	pixel_7_0	pixel_7_1	pixel_7_2	pixel_7_3
0	0.0	0.0	5.0	13.0	9.0	1.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	6.0	0.0
1	0.0	0.0	0.0	12.0	13.0	5.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	4.0	15.0	12.0	0.0	0.0	0.0	0.0	...	5.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	7.0	15.0	13.0	1.0	0.0	0.0	0.0	8.0	...	9.0	0.0	0.0	0.0	0.0	7.0
4	0.0	0.0	0.0	1.0	11.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 64 columns



```
df.describe()
```

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	pixel_0_6	pixel_0_7	pixel_1_0	pixel_1_1	...	pixel_6_6	pixel_6_7	pixel_7_0	pixel_7_1	pixel_7_2	pixel_7_3
count	1797.0	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	...	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000	1797.000000
mean	0.0	0.303840	5.204786	11.835838	11.848080	5.781859	1.362270	0.129661	0.005565	1.993879	...	3.725097	0.206455	0.000000	0.000000	0.000000	0.000000
std	0.0	0.907192	4.754826	4.248842	4.287388	5.666418	3.325775	1.037383	0.094222	3.196160	...	4.919406	0.984401	0.000000	0.000000	0.000000	0.000000
min	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.0	0.000000	1.000000	10.000000	10.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.0	0.000000	4.000000	13.000000	13.000000	4.000000	0.000000	0.000000	0.000000	0.000000	...	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.0	0.000000	9.000000	15.000000	15.000000	11.000000	0.000000	0.000000	0.000000	3.000000	...	7.000000	0.000000	0.000000	0.000000	0.000000	0.000000
max	0.0	8.000000	16.000000	16.000000	16.000000	16.000000	16.000000	15.000000	2.000000	16.000000	...	16.000000	13.000000	0.000000	0.000000	0.000000	0.000000

8 rows × 64 columns

```
X=df    # All features (64 pixels)
y=dataset.target # The actual value (0 → 9)
```

```
# Scaling: each pixel is within the same range
scaler = StandardScaler()
X = scaler.fit_transform(X)
X
```

```
array([[ 0.          , -0.33501649, -0.04308102, ..., -1.14664746,
        -0.5056698 , -0.19600752],
       [ 0.          , -0.33501649, -1.09493684, ...,  0.54856067,
        -0.5056698 , -0.19600752],
       [ 0.          , -0.33501649, -1.09493684, ...,  1.56568555,
        1.6951369 , -0.19600752],
       ...,
       [ 0.          , -0.33501649, -0.88456568, ..., -0.12952258,
        -0.5056698 , -0.19600752],
       [ 0.          , -0.33501649, -0.67419451, ...,  0.8876023 ,
        -0.5056698 , -0.19600752],
       [ 0.          , -0.33501649,  1.00877481, ...,  0.8876023 ,
        -0.26113572, -0.19600752]], shape=(1797, 64))
```


Train a simple Logistic Regression classifier

```
# Data Splitting
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=30)
```

```
# The Model
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
model.score(X_test, y_test)
```

0.9722222222222222

Very Excellent

With 100% information (64 features), we can achieve 97.2% accuracy.

Use PCA with 0.95 for Analysis

```
# PCA (95% Variance): keep 95% of the information
from sklearn.decomposition import PCA
pca = PCA(0.95)
X_pca = pca.fit_transform(X)
X_pca.shape # Instead of 64 features, there are now only 40
```

```
(1797, 40)
```

```
pca.explained_variance_ratio_
# Each number = Importance of each component
```

```
array([0.12033916, 0.09561054, 0.08444415, 0.06498408, 0.04860155,
       0.0421412 , 0.03942083, 0.03389381, 0.02998221, 0.02932003,
       0.02781805, 0.02577055, 0.02275303, 0.0222718 , 0.02165229,
       0.01914167, 0.01775547, 0.01638069, 0.0159646 , 0.01489191,
       0.0134797 , 0.01271931, 0.01165837, 0.01057647, 0.00975316,
       0.00944559, 0.00863014, 0.00836643, 0.00797693, 0.00746471,
       0.00725582, 0.00691911, 0.00653909, 0.00640793, 0.00591384,
       0.00571162, 0.00523637, 0.00481808, 0.00453719, 0.00423163])
```

```
pca.n_components_
```

```
np.int64(40)
```

```
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=30)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_pca, y_train)
model.score(X_test_pca, y_test)
```

```
0.9638888888888889
```

Excellent

We retained 95% of the information (40 features) and achieved 96.4% accuracy.

Use PCA with two features for analysis

```
# PCA (2 features only)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
X_pca.shape
```

```
(1797, 2)
```

```
pca.explained_variance_ratio_
```

```
array([0.12033916, 0.09561054])
```

```
pca.n_components_
```

```
2
```

```
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=30)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_pca, y_train)
model.score(X_test_pca, y_test)
```

```
0.5666666666666667
```

Very Bad

With two features, we retained only 21% and achieved 56.7% accuracy.

Task

Using Digits Dataset:

- Train a model using Logistic Regression
- Using PCA at 80%
- Using PCA with 10 Features
- Comparing Accuracy



Thank You

